

Azure Key Vault – Key Rotation vs Secret Rotation

1. Key vs Secret – Quick Recap

What is a Key?

A **Key** in Azure Key Vault is mainly used for **cryptographic operations**.

Examples:

- Encryption
- Decryption
- Signing
- Verification

Real-time Example

Suppose we have an **Azure Storage Account**.

We can configure it to use a **Customer Managed Key (CMK)** stored in Azure Key Vault to encrypt the storage account.

Flow:

Azure Storage Account → Customer Managed Key → Azure Key Vault

So, a Key is used when we need to perform a cryptographic operation.

2. What is a Secret?

A **Secret** is used to securely store sensitive information such as:

- Connection strings
- Passwords
- API keys
- Application secrets
- Access tokens

Example

Suppose our application needs to connect to SQL Server.

Instead of keeping the connection string directly inside the application configuration, we can store it in Azure Key Vault as a Secret.

Example:

```
Server=MyServer;  
Database=MyDatabase;  
User Id=admin;  
Password=*****;
```

The application retrieves this Secret from Azure Key Vault when required.

3. Key Expiry vs Secret Expiry

This is an important difference.

Key Expiry

When a Key version expires, that **specific key version cannot be used for cryptographic operations**.

For example:

```
Key: StorageEncryptionKey  
  
Version 1  
Expiry: 30-Apr-2025
```

After Version 1 expires, that version can no longer be used for the applicable cryptographic operation.

Important Point

An **older version of a Key can still be used if that version has not expired** and the consuming service is still configured to use it.

Example:

```
Key  
|  
├── Version 1 → Expiry: 30-Apr-2025
```

|
└─ Version 2 → Expiry: 30-Apr-2027

If the Storage Account is still using Version 1 and Version 1 has not expired, it can continue working with Version 1.

4. Secret Expiry

Secret expiry works differently.

If a Secret's expiration date has passed, the Secret is **not automatically deleted or made inaccessible** simply because it is expired.

The application may still be able to retrieve and use the Secret.

Example:

Secret: SqlConnectionString

Expiry Date: 01-Mar-2025

Current Date: 01-Apr-2025

Even though the Secret is expired, the application may still retrieve the Secret value.

Then why configure Secret Expiry?

Secret expiry is mainly useful for:

- Security best practices
- Compliance
- Periodic credential rotation
- Reducing the lifetime of credentials
- Reminding us to replace sensitive values

Think about a banking password.

A bank may ask us to change our password every 3 or 6 months.

Similarly, Secret expiry encourages us to regularly rotate sensitive credentials.

5. What Does Rotation Mean?

Rotation means creating a new version of a Key or Secret.

For example:

```
Key
|
├─ Version 1
└─ Version 2 ← New version
```

Creating Version 2 means we have rotated the Key.

Similarly:

```
Secret
|
├─ Version 1
└─ Version 2 ← New version
```

Creating Version 2 means we have rotated the Secret.

However, the actual process of rotating a Key and rotating a Secret is very different.

6. Key Rotation

Key rotation is relatively straightforward.

We create a **new version of the Key before the existing version expires.**

Example

Suppose:

```
Current Key Version
Expiry: 30-Apr-2025
```

Today:

01-Apr-2025

There are 30 days remaining.

We create a new version:

Version 1
Expiry: 30-Apr-2025

Version 2
Expiry: 01-Apr-2027

Now we have overlapping valid versions.

The old version can continue to work until it expires.

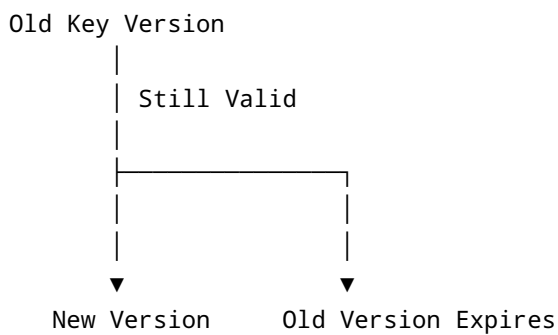
The new version can then become the latest version used by the consuming service.

7. Why Should We Rotate a Key Before Expiry?

The most important reason is to avoid downtime.

We should **not wait until the current Key version expires**.

Instead:



There should be an **overlap period** where both versions are valid.

Good Practice

Create the new Key version well before the current version expires.

The video recommends rotating the Key at least **30 days in advance** as a good practice.

8. Azure Storage Account and Automatic Key Rotation

Suppose an Azure Storage Account is encrypted using a Customer Managed Key from Azure Key Vault.

Initially:

```
Storage Account
  ↓
Key Version 1
```

Now we create:

```
Key Version 2
```

The Storage Account may not immediately switch to Version 2.

When automatic key rotation is enabled, the service periodically checks for a newer Key version and updates to the latest version.

The important concept is:

```
Key Vault
  |
  ├── Version 1
  └── Version 2 ← Latest
      ↑
Storage Account eventually updates
to the latest version
```

The old version can continue to work while it is still valid.

9. Secret Rotation

Secret rotation is **more complicated than Key rotation**.

Why?

Because a Secret often contains a real credential such as:

- SQL connection string
- Password
- API key
- Storage credential

We cannot simply create a new random Secret value.

Example

Suppose our Secret contains:

```
SQL Connection String
Server=MySQLServer;
Database=EmployeeDB;
User Id=admin;
Password=ABC123;
```

If we simply create a new Secret version with:

```
XYZABC
```

the application will fail because `XYZABC` is not a valid SQL connection string.

Therefore, we need to actually generate/update the credential in the target system first.

10. Key Rotation vs Secret Rotation

Key Rotation

Key rotation is comparatively simple.

Basic process:

```
Create new Key version
↓
New version becomes latest
```

↓
Application/service starts using latest version

The important requirement is to create the new version **before the old version expires**.

Secret Rotation

Secret rotation requires coordination with the system that owns the credential.

For example, if the Secret contains a SQL connection string:

```
SQL Database
  ↓
Generate/Create new credential
  ↓
Update Key Vault Secret
  ↓
Application retrieves new Secret
```

We cannot just put a random value into the new Secret version.

11. Example of Secret Rotation

Suppose Key Vault contains:

```
Secret:
SQLConnectionString

Version 1:
Server=SQLServer;
Database=EmployeeDB;
User=Admin;
Password=OldPassword;
```

Before Version 1 expires, we need to:

Step 1 – Generate a new credential

Create a new password/connection string in the SQL Database.


```
New Password = NewPassword123
```

Step 2 – Update the SQL Database

The database now accepts the new credential.

Step 3 – Create a new Secret version

Store the new connection string in Key Vault:

```
Version 2:  
Server=SQLServer;  
Database=EmployeeDB;  
User=Admin;  
Password=NewPassword123;
```

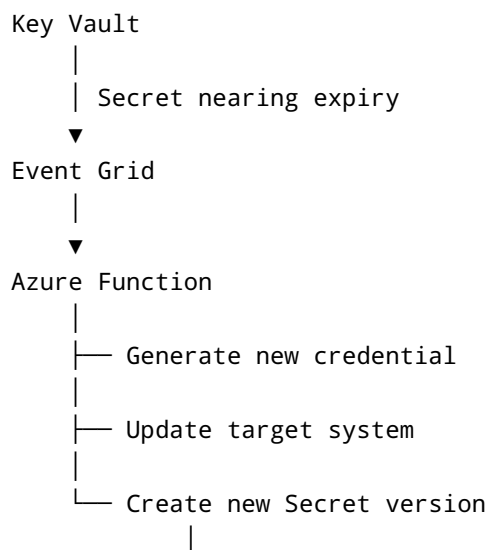
Step 4 – Application uses the new Secret

The application retrieves Version 2 and connects using the new credential.

12. Automated Secret Rotation

Secret rotation can be automated.

A common approach is:



▼
Application

Process

1. A Secret is approaching its expiry.
2. An event can trigger the rotation process.
3. Azure Function is invoked.
4. Azure Function generates a new credential.
5. The credential is updated in the target system such as SQL Database.
6. The new credential is stored as a new Secret version in Azure Key Vault.
7. The application retrieves the new Secret.

This allows us to automate Secret rotation.

13. Key Rotation vs Secret Rotation – Simple Comparison

Feature	Key Rotation	Secret Rotation
Purpose	Cryptographic operations	Store sensitive values
Example	Encryption key	SQL connection string
Rotation	Create new Key version	Create new Secret version
Can we use random new value?	Key is generated/managed as a cryptographic key	No, it must be a valid credential/value
Complexity	Relatively simple	More complex
Target system update required	Usually handled by consuming service	Often yes
Expiry impact	Expired version cannot be used for applicable cryptographic operations	Expiry alone does not necessarily prevent retrieval/use
Automation	Supported by consuming Azure services	Usually requires a rotation workflow
Common tools	Key Vault + Azure service	Key Vault + Event Grid + Azure Function + target system

14. Most Important Interview Points

Q1. What is a Key in Azure Key Vault?

A Key is used for **cryptographic operations** such as encryption, decryption, signing, and verification.

Q2. What is a Secret?

A Secret is used to securely store sensitive values such as passwords, connection strings, API keys, and tokens.

Q3. What is Key rotation?

Key rotation means **creating a new version of an existing Key**.

Q4. What is Secret rotation?

Secret rotation means **creating a new version of a Secret with a new valid credential/value**.

Q5. Why is Secret rotation more complicated?

Because the Secret usually represents a real credential used by another system.

For example, if Key Vault stores a SQL connection string, we must first create/update the credential in SQL and then store the new connection string in Key Vault.

Q6. Does Secret expiry automatically make the Secret unusable?

No. Expiry metadata does not by itself mean the Secret is automatically deleted or inaccessible.

Q7. Can an older Key version still be used?

Yes, if it has not expired and the consuming service is still using that version.

Q8. Why rotate Keys before they expire?

To avoid service disruption and provide an overlap period between the old and new Key versions.

Q9. Why rotate Secrets?

Mainly for:

- Security
- Compliance
- Credential lifecycle management
- Reducing the lifetime of credentials

15. Easy Way to Remember

Remember this simple rule:

KEY = Cryptography

Key → Encrypt / Decrypt / Sign / Verify

SECRET = Sensitive Information

Secret → Password / Connection String / API Key

ROTATION = New Version

Rotation → Create New Version

But:

Key Rotation
= Create new Key version

Whereas:

Secret Rotation
= Create/update real credential
+ Create new Secret version

16. One Real-Time Example

Imagine an application connecting to SQL Database.

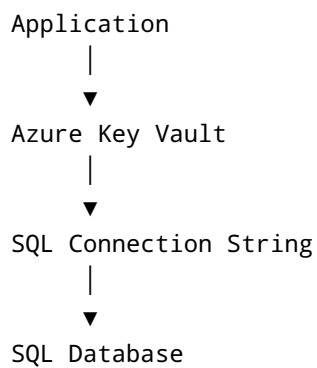
Without Key Vault



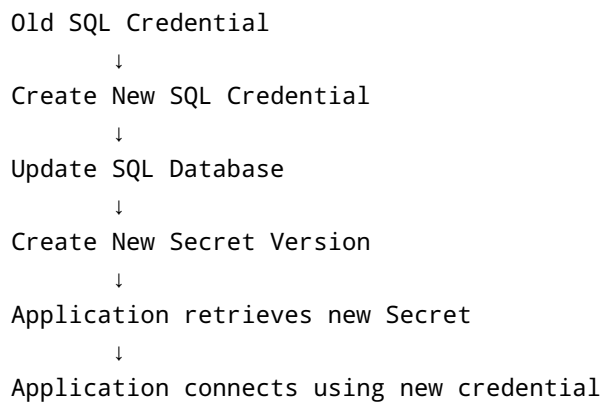
The connection string is stored directly in configuration.

This is not ideal because sensitive information is inside the application configuration.

With Key Vault



When rotating the Secret



This is why **Secret rotation requires more steps than Key rotation.**

17. Final Takeaway

The easiest way to understand the complete concept is:

Key:

Used for cryptographic operations.

Secret:

Used to securely store sensitive values.

Expiry:

Key expiry affects whether that Key version can be used for applicable cryptographic operations. Secret expiry is metadata and does not by itself prevent retrieval/use.

Rotation:

Creating a new version.

Key Rotation:

Create a new Key version before the old one expires and allow the consuming service to move to the latest version.

Secret Rotation:

Generate/update the actual credential in the target system and then store the new credential as a new Secret version.

One-line Interview Answer

"Key rotation is mainly about creating a new cryptographic key version before the old one expires, whereas Secret rotation involves changing the actual credential stored in the Secret and updating the Secret with the new value."